mechanics-ai999x: capability guide

Creator / engineer: Ibnul Jaif Farabi · **Company:** AI999X. Ask the package `Who created you?` or call `mechanics_ai999x.about()` for the same answer.

Every capability mechanics-ai999x supports, generated from its certified capability contracts. Every example below was executed when this guide was built. The structured call `solve("<capability id>", ...)` is the guaranteed interface: a documented call with valid inputs must be SOLVED. Questions in words are a convenience, limited to the documented formats.

Quick start

```
import mechanics_ai999x

result = mechanics_ai999x.solve('damped_angular_frequency', k='1 N/m', m='1 kg', b='1 kg/s')
result.show()
# angular frequency of the damped oscillation = 4.89898 rad/s
```

By topic

topic	capabilities
Collisions	Final velocity of the first body in a one-dimensional elastic collision, Final velocity of the second body in a one-dimensional elastic collision
Orbits	Orbital speed on an elliptical orbit (vis-viva), Escape speed, Orbital period from the semi-major axis (Kepler's third law)
Oscillations	Angular frequency of a damped oscillator, Damping ratio, Quality factor of a damped oscillator, Steady-state amplitude of a driven damped oscillator, Amplitude-resonance frequency of a driven damped oscillator, Higher normal-mode frequency of two coupled oscillators, Period of a physical pendulum, Lower normal mode of a double pendulum, Higher normal mode of a double pendulum
Rigid body	Moment of inertia of a solid sphere, Moment of inertia of a thin spherical shell, Moment of inertia of a solid disk about its axis, Moment of inertia of a thin rod about its center, Moment of inertia of a thin rod about one end, Acceleration of a body rolling without slipping down an incline, Acceleration of an Atwood machine with a massive pulley, Body-frame precession rate of a torque-free symmetric body
Rotating frames	Precession period of a Foucault pendulum, Off-axis equilibrium of a bead on a rotating hoop
Variable mass	Velocity change of a rocket (Tsiolkovsky)

Outcomes

outcome	meaning	what to do
SOLVED	a value with its unit, steps, assumptions and verification	use it; read the assumptions
NEEDS_INPUT	a required input is missing; `uncertainty['needs']` names it	add it (with its unit) and ask again

outcome	meaning	what to do
AMBIGUOUS	two readings or two physical branches give different answers	say which situation is meant
INVALID_INPUT	a unit of the wrong kind, an unknown input name, values with no valid solution	correct the input
OUT_OF_SCOPE	no certified capability computes this (or the structure is not modelled yet)	see the catalogue below
UNRECOGNIZED_FORMAT	the wording was not recognised; a supported template is offered in `uncertainty['try']`	use the template or solve()
INTERNAL_ERROR	a defect of the engine, never a property of the question	report it with the question

Result fields

field	meaning
value, unit	the result in SI units
values	every output of a procedure (x1, x2, discriminant ...)
outcome	one of the seven outcomes above
steps	each equation used, its inputs (with where each came from) and its result
assumptions	conditions and declared defaults the answer relies on
verification	residual of every equation, dimensional consistency, independent checks
latex()	the derivation as LaTeX
show()	renders in a notebook, prints in a terminal

Common mistakes

- Giving a unit of the wrong kind (a mass in m): INVALID_INPUT names the expected unit.
- Leaving out an input: NEEDS_INPUT names it; declared constants (g, k, h ...) are filled and stated.
- Giving every quantity of an equation: say which one to compute with target=.
- Asking several quantities in one question: ask one at a time (or call solve() once per quantity).
- Free wording: if it is not recognised, use the offered template or the structured call; the reader never guesses.

Catalogue

id	operation	family	inputs
damped_angular_frequency	Angular frequency of a damped oscillator	oscillations	wd, k, m, b
damping_ratio	Damping ratio	oscillations	zeta, b, k, m
quality_factor	Quality factor of a damped oscillator	oscillations	Qf, k, m, b
driven_amplitude	Steady-state amplitude of a driven damped oscillator	oscillations	A, F0, k, m, b, w
amplitude_resonance_frequency	Amplitude-resonance frequency of a driven damped oscillator	oscillations	wr, k, m, b
coupled_fast_mode	Higher normal-mode frequency of two coupled oscillators	oscillations	w2, k, kc, m
physical_pendulum_period	Period of a physical pendulum	oscillations	T, Ir, m, g, d

id	operation	family	inputs
double_pendulum_slow_mode	Lower normal mode of a double pendulum	oscillations	w, g, L
double_pendulum_fast_mode	Higher normal mode of a double pendulum	oscillations	w, g, L
vis_viva	Orbital speed on an elliptical orbit (vis-viva)	orbits	v, G, M, r, a
escape_speed	Escape speed	orbits	v, G, M, r
kepler_period_semi_major	Orbital period from the semi-major axis (Kepler's third law)	orbits	T, a, G, M
inertia_solid_sphere	Moment of inertia of a solid sphere	rigid_body	Ir, m, R
inertia_hollow_sphere	Moment of inertia of a thin spherical shell	rigid_body	Ir, m, R
inertia_solid_disk	Moment of inertia of a solid disk about its axis	rigid_body	Ir, m, R
inertia_rod_center	Moment of inertia of a thin rod about its center	rigid_body	Ir, m, Lr
inertia_rod_end	Moment of inertia of a thin rod about one end	rigid_body	Ir, m, Lr
rolling_incline_acceleration	Acceleration of a body rolling without slipping down an incline	rigid_body	a, g, theta, Ir, m, R
atwood_massive_pulley	Acceleration of an Atwood machine with a massive pulley	rigid_body	a, m1, m2, g, Ir, R
torque_free_precession	Body-frame precession rate of a torque-free symmetric body	rigid_body	Wp, w3, I1, I3
foucault_precession_period	Precession period of a Foucault pendulum	rotating_frames	T, W, phi
bead_hoop_equilibrium	Off-axis equilibrium of a bead on a rotating hoop	rotating_frames	theta, g, w, R
elastic_collision_first_final	Final velocity of the first body in a one-dimensional elastic collision	collisions	v1f, m1, m2, v1, v2
elastic_collision_second_final	Final velocity of the second body in a one-dimensional elastic collision	collisions	v2f, m1, m2, v1, v2
rocket_delta_v	Velocity change of a rocket (Tsiolkovsky)	variable_mass	dv, ve, m0, mf

Angular frequency of a damped oscillator — `damped_angular_frequency`

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$

input	meaning	kind / role	unit	default
`wd`	angular frequency of the damped oscillation	angular_frequency / damped	`rad/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** damped
- **Applies when:** a mass on a linear spring with viscous damping
- **Fails when:** critical or over-damping
- **Assumes:** linear viscous damping; underdamped (b^2 < 4 k m)
- **Evidence:** theorem

```
mechanics_ai999x.solve('damped_angular_frequency', k='50 N/m', m='2 kg', b='4 kg/s', target='wd')
# -> 4.89898 rad/s   (SOLVED)
```

$$\omega_d = \sqrt{\frac{k}{m} - \frac{b^2}{4m^2}}$$
$$\omega_d = \frac{\sqrt{-b^2 + 4km}}{2m} = 4.89898\ \mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	damped_angular_frequency needs k (spring constant, N/m). Example: solve('damped_angular_frequency', k='... N/m', m='... kg', b='... kg/s').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

- "A 2 kg mass on a spring with a spring constant of 50 N/m has a damping coefficient of 4 kg/s. What is the damped angular frequency?" → 4.89898 rad/s
- "A damped oscillator has mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s. Find its damped angular frequency." → 3 rad/s

Damping ratio — `damping_ratio`

$$\zeta = \frac{b}{2\sqrt{km}}$$

input	meaning	kind / role	unit	default
`zeta`	damping ratio	damping_ratio / plain	`1`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	

- **Conditions:** damped
- **Applies when:** a damped mass-spring system
- **Fails when:** nonlinear damping
- **Assumes:** linear viscous damping
- **Evidence:** theorem

```
mechanics_ai999x.solve('damping_ratio', b='4 kg/s', k='50 N/m', m='2 kg', target='zeta')
# -> 0.2 1   (SOLVED)
```

$$\zeta = \frac{b}{2\sqrt{km}}$$
$$\zeta = \frac{b}{2\sqrt{k}\sqrt{m}} = 0.2\ 1$$

invalid call	outcome	message
without `b`	NEEDS_INPUT	damping_ratio needs b (viscous damping coefficient, kg/s). Example: solve('damping_ratio', b='... kg/s', k='... N/m', m='... kg').
`b` in a wrong unit	INVALID_INPUT	b is a damping: its unit must be like kg/s, not 's'.

Questions in words that work:

- "A damped oscillator has a 2 kg mass, a spring constant of 50 N/m and a damping coefficient of 4 kg/s. What is the damping ratio?" → 0.2 1
- "Find the damping ratio of a damped spring system with mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s." → 0.316228 1

Quality factor of a damped oscillator — `quality_factor`

$$Q_f = \frac{\sqrt{km}}{b}$$

input	meaning	kind / role	unit	default
`Qf`	quality factor	quality_factor / plain	`1`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** damped
- **Applies when:** a damped mass-spring system
- **Fails when:** nonlinear damping
- **Assumes:** linear viscous damping
- **Evidence:** theorem

```
mechanics_ai999x.solve('quality_factor', k='50 N/m', m='2 kg', b='4 kg/s', target='Qf')
# -> 2.5 1   (SOLVED)
```

$$Q_f = \frac{\sqrt{km}}{b}$$
$$Q_f = \frac{\sqrt{k}\sqrt{m}}{b} = 2.5\ 1$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	quality_factor needs k (spring constant, N/m). Example: solve('quality_factor', k='... N/m', m='... kg', b='... kg/s').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

- "What is the quality factor of a damped oscillator with mass 2 kg, spring constant 50 N/m and damping coefficient 4 kg/s?" → 2.5 1
- "A damped oscillator has a 1 kg mass, a spring constant of 100 N/m and a damping coefficient of 0.5 kg/s. Find its quality factor." → 20 1

Steady-state amplitude of a driven damped oscillator — `driven_amplitude`

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$

input	meaning	kind / role	unit	default
`A`	steady-state amplitude	length / amplitude	`m`	
`F0`	driving force amplitude	force / driving	`N`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	
`w`	driving angular frequency	angular_frequency / driving	`rad/s`	

- **Conditions:** driven
- **Applies when:** a damped mass-spring system driven by a sinusoidal force
- **Fails when:** before the transients decay
- **Assumes:** sinusoidal driving force; linear viscous damping; steady state (transients have died out)
- **Evidence:** theorem

```
mechanics_ai999x.solve('driven_amplitude', F0='10 N', k='50 N/m', m='2 kg', b='4 kg/s', w='3
rad/s', target='A')
# -> 0.292603 m   (SOLVED)
```

$$A = \frac{F_0}{\sqrt{(k - m\omega^2)^2 + (b\omega)^2}}$$
$$A = \frac{F_0}{\sqrt{b^2\omega^2 + (k - m\omega^2)^2}} = 0.292603\ \mathrm{m}$$

invalid call	outcome	message
without `F0`	NEEDS_INPUT	driven_amplitude needs F0 (driving force amplitude, N). Example: solve('driven_amplitude', F0='... N', k='... N/m', m='... kg', b='... kg/s'
`F0` in a wrong unit	INVALID_INPUT	F0 is a force: its unit must be like N, not 'kg'.

Questions in words that work:

- "A 2 kg mass on a spring with spring constant 50 N/m and damping coefficient 4 kg/s is driven by a periodic force of amplitude 10 N at a driving frequency of 3 rad/s. What is the steady-state amplitude?" → 0.292603 m

- "A driven oscillator of mass 1 kg has spring constant 20 N/m and damping coefficient 1 kg/s. The driving force amplitude is 2 N and the driving frequency is 4 rad/s. Find the steady-state amplitude." → 0.353553 m

Amplitude-resonance frequency of a driven damped oscillator — `amplitude_resonance_frequency`

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$

input	meaning	kind / role	unit	default
`wr`	driving frequency of maximum amplitude	angular_frequency / resonance	`rad/s`	
`k`	spring constant	spring_constant / plain	`N/m`	
`m`	oscillating mass	mass / plain	`kg`	
`b`	viscous damping coefficient	damping / plain	`kg/s`	

- **Conditions:** driven
- **Applies when:** the driving frequency that maximises the steady-state amplitude
- **Fails when:** $b^2 \geq 2$ k m (no resonance peak)
- **Assumes:** sinusoidal driving; linear viscous damping; $b^2 < 2$ k m
- **Evidence:** theorem

```
mechanics_ai999x.solve('amplitude_resonance_frequency', k='50 N/m', m='2 kg', b='4 kg/s',
target='wr')
# -> 4.79583 rad/s   (SOLVED)
```

$$\omega_r = \sqrt{\frac{k}{m} - \frac{b^2}{2m^2}}$$
$$\omega_r = \frac{\sqrt{-2b^2 + 4km}}{2m} = 4.79583\ \mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	amplitude_resonance_frequency needs k (spring constant, N/m). Example: solve('amplitude_resonance_frequency', k='... N/m', m='... kg', b='..
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

- "A driven oscillator has mass 2 kg, spring constant 50 N/m and damping coefficient 4 kg/s. What is its amplitude resonance frequency?" → 4.79583 rad/s
- "Find the amplitude resonance frequency of a driven damped oscillator with mass 1 kg, spring constant 10 N/m and damping coefficient 2 kg/s." → 2.82843 rad/s

Higher normal-mode frequency of two coupled oscillators — `coupled_fast_mode`

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$

input	meaning	kind / role	unit	default
`w2`	higher normal-mode angular frequency	angular_frequency / mode_fast	`rad/s`	
`k`	each outer spring's constant	spring_constant / plain	`N/m`	
`kc`	coupling spring constant	spring_constant / coupling	`N/m`	
`m`	each mass	mass / plain	`kg`	

- **Conditions:** coupled
- **Applies when:** two equal masses joined by a coupling spring, each also tied to a wall
- **Fails when:** unequal masses or springs
- **Assumes:** two identical masses; identical outer springs; small oscillations, no damping
- **Evidence:** theorem

```
mechanics_ai999x.solve('coupled_fast_mode', k='40 N/m', kc='10 N/m', m='2 kg', target='w2')
# -> 5.47723 rad/s   (SOLVED)
```

$$\omega_2 = \sqrt{\frac{k + 2k_c}{m}}$$
$$\omega_2 = \frac{\sqrt{k + 2k_c}}{\sqrt{m}} = 5.47723\ \mathrm{rad/s}$$

invalid call	outcome	message
without `k`	NEEDS_INPUT	coupled_fast_mode needs k (each outer spring's constant, N/m). Example: solve('coupled_fast_mode', k='... N/m', kc='... N/m', m='... kg').
`k` in a wrong unit	INVALID_INPUT	k is a spring constant: its unit must be like N/m, not 'kg'.

Questions in words that work:

- "Two identical masses of 2 kg are each attached to a wall by a spring of spring constant 40 N/m and joined by a coupling spring of 10 N/m. What is the higher normal mode frequency?" → 5.47723 rad/s
- "Two coupled oscillators have masses of 1 kg each, outer spring constant 8 N/m and coupling spring constant 4 N/m. Find the higher normal-mode frequency." → 4 rad/s

Period of a physical pendulum — `physical_pendulum_period`

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

input	meaning	kind / role	unit	default
`T`	small-oscillation period	time / period	`s`	

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the pivot	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the body	mass / plain	`kg`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`d`	distance from the pivot to the center of mass	length / pivot_cm	`m`	

- **Conditions:** physical_pendulum
- **Applies when:** a rigid body swinging about a fixed pivot
- **Fails when:** large amplitudes
- **Assumes:** small oscillations; no friction at the pivot
- **Evidence:** theorem

```
mechanics_ai999x.solve('physical_pendulum_period', Ir='0.5 kg*m^2', m='2 kg', d='0.3 m',
target='T')
# -> 1.83159 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{I_r}{mgd}}$$

$$T = \frac{2\pi\sqrt{I_r}}{\sqrt{d}\sqrt{g}\sqrt{m}} = 1.83159\ \text{s}$$

invalid call	outcome	message
without `Ir`	NEEDS_INPUT	physical_pendulum_period needs Ir (moment of inertia about the pivot, kg*m^2). Example: solve('physical_pendulum_period', Ir='... kg*m^2', m
`Ir` in a wrong unit	INVALID_INPUT	Ir is a moment of inertia: its unit must be like kg*m^2, not 's'.

Questions in words that work:

- "A physical pendulum of mass 2 kg has a moment of inertia of 0.5 kg*m^2 about its pivot, and the distance from the pivot to the center of mass is 0.3 m. What is its period?" → 1.83159 s
- "Find the period of a compound pendulum of mass 3 kg with moment of inertia 1.2 kg*m^2 and a distance from the pivot to the center of mass of 0.25 m." → 2.53793 s

Lower normal mode of a double pendulum — `double_pendulum_slow_mode`

$$\omega = \sqrt{\frac{\left(2-\sqrt{2}\right)g}{L}}$$

input	meaning	kind / role	unit	default
`w`	lower normal-mode angular frequency	angular_frequency / mode_slow	`rad/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`L`	length of each arm	length / pendulum	`m`	

- **Conditions:** double_pendulum

- **Applies when:** a double pendulum with equal masses and arms
- **Fails when:** large amplitudes (chaotic motion)
- **Assumes:** equal masses and arm lengths; small oscillations
- **Evidence:** theorem

```
mechanics_ai999x.solve('double_pendulum_slow_mode', L='1 m', target='w')
# -> 2.39679 rad/s   (SOLVED)
```

$$\omega = \sqrt{\frac{\left(2-\sqrt{2}\right)g}{L}}$$

$$\omega = \frac{\sqrt{g}\sqrt{2-\sqrt{2}}}{\sqrt{L}} = 2.39679\ \mathrm{rad/s}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	double_pendulum_slow_mode needs L (length of each arm, m). Example: solve('double_pendulum_slow_mode', g='... m/s^2', L='... m').
`L` in a wrong unit	INVALID_INPUT	L is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "A double pendulum has equal masses and a length of each arm of 1 m. What is the lower normal mode frequency?" → 2.39679 rad/s
- "Find the lower normal-mode frequency of a double pendulum whose arm length is 0.5 m." → 3.38957 rad/s

Higher normal mode of a double pendulum — `double_pendulum_fast_mode`

$$\omega = \sqrt{\frac{\left(2+\sqrt{2}\right)g}{L}}$$

input	meaning	kind / role	unit	default
`w`	higher normal-mode angular frequency	angular_frequency / mode_fast	`rad/s`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`L`	length of each arm	length / pendulum	`m`	

- **Conditions:** double_pendulum
- **Applies when:** a double pendulum with equal masses and arms
- **Fails when:** large amplitudes (chaotic motion)
- **Assumes:** equal masses and arm lengths; small oscillations
- **Evidence:** theorem

```
mechanics_ai999x.solve('double_pendulum_fast_mode', L='1 m', target='w')
# -> 5.78636 rad/s   (SOLVED)
```

$$\omega = \sqrt{\frac{\left(2+\sqrt{2}\right)g}{L}}$$

$$\omega = \frac{\sqrt{g}\sqrt{\sqrt{2}+2}}{\sqrt{L}} = 5.78636\ \mathrm{rad/s}$$

invalid call	outcome	message
without `L`	NEEDS_INPUT	double_pendulum_fast_mode needs L (length of each arm, m). Example: solve('double_pendulum_fast_mode', g='... m/s^2', L='... m').
`L` in a wrong unit	INVALID_INPUT	L is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "A double pendulum has equal masses and a length of each arm of 1 m. What is the higher normal mode frequency?" → 5.78636 rad/s
- "Find the higher normal-mode frequency of a double pendulum whose arm length is 0.5 m." → 8.18315 rad/s

Orbital speed on an elliptical orbit (vis-viva) — `vis_viva`

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

input	meaning	kind / role	unit	default
`v`	orbital speed at distance r	speed / orbital	`m/s`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the central body	mass / plain	`kg`	
`r`	distance from the center of the central body	length / radius	`m`	
`a`	semi-major axis	length / semi_major	`m`	

- **Conditions:** orbit
- **Applies when:** a satellite or planet on an elliptical orbit
- **Fails when:** unbound orbits; comparable masses
- **Assumes:** two bodies, the orbiting one much lighter; a bound (elliptical) orbit
- **Evidence:** theorem

```
mechanics_ai999x.solve('vis_viva', M='5.972e+24 kg', r='7e+06 m', a='8e+06 m', target='v')
# -> 8003.69 m/s   (SOLVED)
```

$$v = \sqrt{GM\left(\frac{2}{r} - \frac{1}{a}\right)}$$

$$v = \frac{\sqrt{G}\sqrt{M}\sqrt{2a - r}}{\sqrt{a}\sqrt{r}} = 8003.69\ \mathrm{m/s}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	vis_viva needs M (mass of the central body, kg). Example: solve('vis_viva', G='... N*m^2/kg^2', M='... kg', r='... m', a='... m').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A satellite on an elliptical orbit around a planet of mass 5.972e24 kg has a semi-major axis of 8.0e6 m. What is its orbital speed at a distance from the center of 7.0e6 m?" → 8003.69 m/s
- "A comet orbits a star of mass 1.989e30 kg with a semi-major axis of 2.0e11 m. Find its orbital speed at a distance from the center of 1.5e11 m." → 33260.6 m/s

Escape speed — `escape_speed`

$$v = \sqrt{\frac{2GM}{r}}$$

input	meaning	kind / role	unit	default
`v`	escape speed	speed / escape	`m/s`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the body	mass / plain	`kg`	
`r`	distance from its center	length / radius	`m`	

- **Conditions:** escape
- **Applies when:** leaving a body's gravity from distance r
- **Fails when:** atmospheric drag; other bodies' gravity
- **Assumes:** no air resistance; a spherically symmetric body; starting from rest relative to it
- **Evidence:** theorem

```
mechanics_ai999x.solve('escape_speed', M='5.972e+24 kg', r='6.371e+06 m', target='v')
# -> 11186 m/s   (SOLVED)
```

$$v = \sqrt{\frac{2GM}{r}}$$

$$v = \frac{\sqrt{2}\sqrt{G}\sqrt{M}}{\sqrt{r}} = 11186\ \mathrm{m/s}$$

invalid call	outcome	message
without `M`	NEEDS_INPUT	escape_speed needs M (mass of the body, kg). Example: solve('escape_speed', G='... N*m^2/kg^2', M='... kg', r='... m').
`M` in a wrong unit	INVALID_INPUT	M is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "What is the escape speed from a planet of mass 5.972e24 kg and radius 6.371e6 m?" → 11186 m/s
- "Find the escape velocity at the surface of a moon with a mass of 7.342e22 kg and a radius of 1.737e6 m." → 2375.34 m/s

Orbital period from the semi-major axis (Kepler's third law) — `kepler_period_semi_major`

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$

input	meaning	kind / role	unit	default
`T`	orbital period	time / period	`s`	
`a`	semi-major axis	length / semi_major	`m`	
`G`		gravitational_constant / constant	`N*m^2/kg^2`	6.6743e-11 N*m^2/kg^2
`M`	mass of the central body	mass / plain	`kg`	

- **Conditions:** orbit
- **Applies when:** any bound elliptical orbit
- **Fails when:** comparable masses (use the total mass)
- **Assumes:** two bodies, the orbiting one much lighter
- **Evidence:** theorem

```
mechanics_ai999x.solve('kepler_period_semi_major', a='1.496e+11 m', M='1.989e+30 kg', target='T')
# -> 3.15542e+07 s   (SOLVED)
```

$$T = 2\pi\sqrt{\frac{a^3}{GM}}$$
$$T = \frac{2\pi a^{\frac{3}{2}}}{\sqrt{G}\sqrt{M}} = 3.15542e+07\ \mathrm{s}$$

invalid call	outcome	message
without `a`	NEEDS_INPUT	kepler_period_semi_major needs a (semi-major axis, m). Example: solve('kepler_period_semi_major', a='... m', G='... N*m^2/kg^2', M='... kg')
`a` in a wrong unit	INVALID_INPUT	a is a length: its unit must be like m, not 'kg'.

Questions in words that work:

- "A planet orbits a star of mass 1.989e30 kg on an elliptical orbit with a semi-major axis of 1.496e11 m. What is its orbital period?" → 3.15542e+07 s

- "Find the orbital period of a satellite on an elliptical orbit around a planet of mass 5.972e24 kg with a semi-major axis of 4.2164e7 m." → 86164.8 s

Moment of inertia of a solid sphere — `inertia_solid_sphere`

$$I_r = \frac{2mR^2}{5}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** solid_sphere
- **Applies when:** a uniform solid sphere
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem

```
mechanics_ai999x.solve('inertia_solid_sphere', m='2 kg', R='0.5 m', target='Ir')
# -> 0.2 kg*m^2   (SOLVED)
```

$$I_r = \frac{2mR^2}{5}$$
$$I_r = \frac{2R^2m}{5} = 0.2\ \mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_solid_sphere needs m (mass, kg). Example: solve('inertia_solid_sphere', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A uniform solid sphere has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.2 kg*m^2
- "Find the moment of inertia of a solid sphere of mass 5 kg and radius 0.2 m about its central axis." → 0.08 kg*m^2

Moment of inertia of a thin spherical shell — `inertia_hollow_sphere`

$$I_r = \frac{2mR^2}{3}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	

input	meaning	kind / role	unit	default
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** hollow_sphere
- **Applies when:** a uniform hollow sphere
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem

```
mechanics_ai999x.solve('inertia_hollow_sphere', m='2 kg', R='0.5 m', target='Ir')
# -> 0.333333 kg*m^2   (SOLVED)
```

$$I_r = \frac{2mR^2}{3}$$
$$I_r = \frac{2R^2m}{3} = 0.333333\ \mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_hollow_sphere needs m (mass, kg). Example: solve('inertia_hollow_sphere', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A uniform hollow sphere has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.333333 kg*m^2
- "Find the moment of inertia of a hollow sphere of mass 5 kg and radius 0.2 m about its central axis." → 0.133333 kg*m^2

Moment of inertia of a solid disk about its axis — `inertia_solid_disk`

$$I_r = \frac{mR^2}{2}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about the central axis	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	radius	length / radius	`m`	

- **Conditions:** solid_disk
- **Applies when:** a uniform solid disk
- **Fails when:** non-uniform density; another axis (use the parallel-axis theorem)
- **Assumes:** uniform density; the axis through the center (the symmetry axis for a disk)
- **Evidence:** theorem

```
mechanics_ai999x.solve('inertia_solid_disk', m='2 kg', R='0.5 m', target='Ir')
# -> 0.25 kg*m^2   (SOLVED)
```

$$I_r = \frac{mR^2}{2}$$
$$I_r = \frac{R^2 m}{2} = 0.25\ \mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_solid_disk needs m (mass, kg). Example: solve('inertia_solid_disk', m='... kg', R='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A uniform solid disk has a mass of 2 kg and a radius of 0.5 m. What is its moment of inertia about its center?" → 0.25 kg*m^2
- "Find the moment of inertia of a solid disk of mass 5 kg and radius 0.2 m about its central axis." → 0.1 kg*m^2

Moment of inertia of a thin rod about its center — `inertia_rod_center`

$$I_r = \frac{mL_r^2}{12}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about its center	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the rod	mass / plain	`kg`	
`Lr`	length of the rod	length / plain	`m`	

- **Conditions:** rod, about_center
- **Applies when:** a thin uniform rod rotating about its center
- **Fails when:** a thick rod; a non-uniform rod
- **Assumes:** a thin uniform rod; the axis perpendicular to the rod
- **Evidence:** theorem

```
mechanics_ai999x.solve('inertia_rod_center', m='3 kg', Lr='2 m', target='Ir')
# -> 1 kg*m^2   (SOLVED)
```

$$I_r = \frac{mL_r^2}{12}$$
$$I_r = \frac{L_r^2 m}{12} = 1\ \mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_rod_center needs m (mass of the rod, kg). Example: solve('inertia_rod_center', m='... kg', Lr='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A uniform rod of mass 3 kg and length 2 m rotates about its center. What is its moment of inertia?" → 1 kg*m^2
- "Find the moment of inertia of a thin rod of mass 0.6 kg and length 1 m about its center." → 0.05 kg*m^2

Moment of inertia of a thin rod about one end — `inertia_rod_end`

$$I_r = \frac{mL_r^2}{3}$$

input	meaning	kind / role	unit	default
`Ir`	moment of inertia about one end	moment_of_inertia / plain	`kg*m^2`	
`m`	mass of the rod	mass / plain	`kg`	
`Lr`	length of the rod	length / plain	`m`	

- **Conditions:** rod, about_end
- **Applies when:** a thin uniform rod rotating about one end
- **Fails when:** a thick rod; a non-uniform rod
- **Assumes:** a thin uniform rod; the axis perpendicular to the rod
- **Evidence:** theorem

```
mechanics_ai999x.solve('inertia_rod_end', m='3 kg', Lr='2 m', target='Ir')
# -> 4 kg*m^2   (SOLVED)
```

$$I_r = \frac{mL_r^2}{3}$$
$$I_r = \frac{L_r^2 m}{3} = 4\,\mathrm{kg\,m^2}$$

invalid call	outcome	message
without `m`	NEEDS_INPUT	inertia_rod_end needs m (mass of the rod, kg). Example: solve('inertia_rod_end', m='... kg', Lr='... m').
`m` in a wrong unit	INVALID_INPUT	m is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "A uniform rod of mass 3 kg and length 2 m rotates about one end. What is its moment of inertia?" → 4 kg*m^2
- "Find the moment of inertia of a thin rod of mass 0.6 kg and length 1 m about one end." → 0.2 kg*m^2

Acceleration of a body rolling without slipping down an incline — `rolling_incline_acceleration`

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$

input	meaning	kind / role	unit	default
`a`	acceleration of the center of mass along the incline	acceleration / plain	`m/s^2`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`theta`	angle of the incline	angle / incline	`rad`	
`Ir`	moment of inertia about the center	moment_of_inertia / plain	`kg*m^2`	
`m`	mass	mass / plain	`kg`	
`R`	rolling radius	length / radius	`m`	

- **Conditions:** rolling
- **Applies when:** a round body rolling down a straight incline
- **Fails when:** slipping; rolling resistance
- **Assumes:** rolling without slipping; no rolling resistance
- **Evidence:** theorem

```
mechanics_ai999x.solve('rolling_incline_acceleration', theta='30 deg', Ir='0.04 kg*m^2', m='2 kg',
R='0.2 m', target='a')
# -> 3.26888 m/s^2   (SOLVED)
```

$$a = \frac{g \sin(\theta)}{1 + \frac{I_r}{mR^2}}$$
$$a = \frac{R^2 g m \sin(\theta)}{I_r + R^2 m} = 3.26888 \ \mathrm{m/s^2}$$

invalid call	outcome	message
without `theta`	NEEDS_INPUT	rolling_incline_acceleration needs theta (angle of the incline, rad). Example: solve('rolling_incline_acceleration', g='... m/s^2', theta='.
`theta` in a wrong unit	INVALID_INPUT	theta is a angle: its unit must be like rad, not 'kg'.

Questions in words that work:

- "A cylinder of mass 2 kg and radius 0.2 m with a moment of inertia of 0.04 kg*m^2 rolls without slipping down an incline. The angle of the incline is 30 degrees. What is its acceleration?" → 3.26888 m/s^2
- "A wheel rolls without slipping down a slope inclined at 20 degrees. It has mass 4 kg, radius 0.5 m and moment of inertia 0.5 kg*m^2. Find its acceleration." → 2.23605 m/s^2

Acceleration of an Atwood machine with a massive pulley — `atwood_massive_pulley`

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$

input	meaning	kind / role	unit	default
`a`	acceleration of each mass	acceleration / plain	`m/s^2`	
`m1`	heavier mass	mass / heavy	`kg`	
`m2`	lighter mass	mass / light	`kg`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`Ir`	moment of inertia of the pulley	moment_of_inertia / plain	`kg*m^2`	
`R`	radius of the pulley	length / radius	`m`	

- **Conditions:** atwood
- **Applies when:** two masses hung over a pulley with rotational inertia
- **Fails when:** the string slips; axle friction
- **Assumes:** a massless string that does not slip on the pulley; a frictionless axle
- **Evidence:** theorem

```
mechanics_ai999x.solve('atwood_massive_pulley', m1='3 kg', m2='2 kg', Ir='0.02 kg*m^2', R='0.1 m',
target='a')
# -> 1.40095 m/s^2   (SOLVED)
```

$$a = \frac{(m_1 - m_2)g}{m_1 + m_2 + \frac{I_r}{R^2}}$$
$$a = \frac{R^2 g\,(m_1 - m_2)}{I_r + R^2 m_1 + R^2 m_2} = 1.40095\ \mathrm{m/s^2}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	atwood_massive_pulley needs m1 (heavier mass, kg). Example: solve('atwood_massive_pulley', m1='... kg', m2='... kg', g='... m/s^2', Ir='...
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "In an Atwood machine the heavier mass is 3 kg and the lighter mass is 2 kg. They hang over a pulley with a moment of inertia of 0.02 kg*m^2 and a radius of 0.1 m. What is the acceleration of the masses?" → 1.40095 m/s^2
- "Find the acceleration of an Atwood machine whose heavier mass is 5 kg and lighter mass is 1 kg, hung over a pulley of radius 0.5 m with a moment of inertia of 0.5 kg*m^2." → 4.90332 m/s^2

Body-frame precession rate of a torque-free symmetric body — `torque_free_precession`

$$\Omega_p = \frac{\omega_3\,(I_3 - I_1)}{I_1}$$

input	meaning	kind / role	unit	default
`Wp`	precession rate seen in the body frame	angular_frequency / precession	`rad/s`	
`w3`	spin angular velocity about the symmetry axis	angular_frequency / spin	`rad/s`	
`I1`	moment of inertia about a transverse axis	moment_of_inertia / transverse	`kg*m^2`	
`I3`	moment of inertia about the symmetry axis	moment_of_inertia / axial	`kg*m^2`	

- **Conditions:** torque_free
- **Applies when:** a symmetric rigid body spinning freely
- **Fails when:** external torques; an asymmetric body
- **Assumes:** a rigid symmetric body $(I_1 = I_2)$; no external torque; an oblate body $(I_3 > I_1)$
- **Evidence:** theorem

```
mechanics_ai999x.solve('torque_free_precession', w3='10 rad/s', I1='2 kg*m^2', I3='3 kg*m^2',
target='Wp')
# -> 5 rad/s   (SOLVED)
```

$$\Omega_p = \frac{\omega_3 \left(I_3 - I_1\right)}{I_1}$$
$$\Omega_p = \frac{\omega_3 \left(-I_1 + I_3\right)}{I_1} = 5\ \mathrm{rad/s}$$

invalid call	outcome	message
without `w3`	NEEDS_INPUT	torque_free_precession needs w3 (spin angular velocity about the symmetry axis, rad/s). Example: solve('torque_free_precession', w3='... rad
`w3` in a wrong unit	INVALID_INPUT	w3 is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

- "A torque-free symmetric body spins with a spin angular velocity of 10 rad/s. Its transverse moment of inertia is 2 kg*m^2 and its moment of inertia about the symmetry axis is 3 kg*m^2. What is the precession rate?" → 5 rad/s
- "In deep space a symmetric rigid body has a spin rate of 4 rad/s, a transverse moment of inertia of 1 kg*m^2 and an axial moment of inertia of 1.5 kg*m^2. Find its precession rate." → 2 rad/s

Precession period of a Foucault pendulum — `foucault_precession_period`

$$T = \frac{2\pi}{\Omega \sin\left(\phi\right)}$$

input	meaning	kind / role	unit	default
`T`	time for the swing plane to turn once	time / precession_period	`s`	
`W`		angular_frequency / rotation	`rad/s`	7.2921159e-05 rad/s
`phi`	latitude	angle / latitude	`rad`	

- **Conditions:** foucault
- **Applies when:** a long pendulum on a rotating planet

- **Fails when:** at the equator (no precession)
- **Assumes:** small oscillations; the rotation much slower than the swing
- **Evidence:** theorem

```
mechanics_ai999x.solve('foucault_precession_period', phi='48.85 deg', target='T')
# -> 114429 s   (SOLVED)
```

$$T = \frac{2\pi}{\Omega \sin(\phi)}$$
$$T = \frac{2\pi}{\Omega \sin(\phi)} = 114429 \text{ s}$$

invalid call	outcome	message
without `phi`	NEEDS_INPUT	foucault_precession_period needs phi (latitude, rad). Example: solve('foucault_precession_period', W='... rad/s', phi='... rad').
`phi` in a wrong unit	INVALID_INPUT	phi is a angle: its unit must be like rad, not 'kg'.

Questions in words that work:

- "A Foucault pendulum swings in Paris at a latitude of 48.85 degrees. What is its precession period?" → 114429 s
- "Find the precession period of a Foucault pendulum at the North Pole, latitude 90 degrees." → 86164.1 s

Off-axis equilibrium of a bead on a rotating hoop — `bead_hoop_equilibrium`

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$

input	meaning	kind / role	unit	default
`theta`	equilibrium angle from the bottom	angle / equilibrium	`rad`	
`g`		acceleration / gravity	`m/s^2`	9.80665 m/s^2
`w`	angular velocity of the hoop	angular_frequency / rotation	`rad/s`	
`R`	radius of the hoop	length / radius	`m`	

- **Conditions:** hoop
- **Applies when:** a bead free to slide on a vertical hoop spinning about a vertical diameter
- **Fails when:** $w^2\ R \leq g$ (the bead stays at the bottom)
- **Assumes:** a frictionless bead; the hoop turning about its vertical diameter; $w^2\ R > g$
- **Evidence:** theorem

```
mechanics_ai999x.solve('bead_hoop_equilibrium', w='10 rad/s', R='0.2 m', target='theta')
# -> 1.05833 rad   (SOLVED)
```

$$\theta = \arccos\left(\frac{g}{\omega^2 R}\right)$$
$$\theta = \operatorname{acos}\left(\frac{g}{R\omega^2}\right) = 1.05833\ \text{rad}$$

invalid call	outcome	message
without `w`	NEEDS_INPUT	bead_hoop_equilibrium needs w (angular velocity of the hoop, rad/s). Example: solve('bead_hoop_equilibrium', g='... m/s^2', w='... rad/s', R
`w` in a wrong unit	INVALID_INPUT	w is a angular frequency: its unit must be like rad/s, not 'kg'.

Questions in words that work:

- "A bead slides without friction on a rotating hoop of radius 0.2 m. The hoop rotates at 10 rad/s about its vertical diameter. What is the equilibrium angle of the bead?" → 1.05833 rad
- "A bead on a hoop of radius 1 m rotating at 5 rad/s about a vertical diameter: find the equilibrium angle." → 1.1677 rad

Final velocity of the first body in a one-dimensional elastic collision — `elastic_collision_first_final`

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2 v_2}{m_1 + m_2}$$

input	meaning	kind / role	unit	default
`v1f`	final velocity of the first body	speed / first_final	`m/s`	
`m1`	mass of the first body	mass / first	`kg`	
`m2`	mass of the second body	mass / second	`kg`	
`v1`	initial velocity of the first body	speed / first_initial	`m/s`	
`v2`	initial velocity of the second body	speed / second_initial	`m/s`	

- **Conditions:** elastic_collision
- **Applies when:** two bodies colliding elastically along a line
- **Fails when:** inelastic collisions; glancing collisions
- **Assumes:** a head-on (one-dimensional) collision; kinetic energy conserved; no external forces
- **Evidence:** theorem

```
mechanics_ai999x.solve('elastic_collision_first_final', m1='2 kg', m2='1 kg', v1='3 m/s', v2='0
m/s', target='v1f')
# -> 1 m/s   (SOLVED)
```

$$v_{1f} = \frac{(m_1 - m_2)v_1 + 2m_2 v_2}{m_1 + m_2}$$
$$v_{1f} = \frac{m_1 v_1 - m_2 v_1 + 2m_2 v_2}{m_1 + m_2} = 1\ \text{m/s}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	elastic_collision_first_final needs m1 (mass of the first body, kg). Example: solve('elastic_collision_first_final', m1='... kg', m2='... kg
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "In an elastic collision the first mass is 2 kg with a velocity of the first of 3 m/s, and the second mass is 1 kg with a velocity of the second of 0 m/s. What is the final velocity of the first?" → 1 m/s
- "Two carts collide elastically: the first mass is 1 kg and the velocity of the first is 4 m/s; the second mass is 3 kg and the velocity of the second is -2 m/s. Find the final velocity of the first." → -5 m/s

Final velocity of the second body in a one-dimensional elastic collision —

`elastic_collision_second_final`

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$

input	meaning	kind / role	unit	default
`v2f`	final velocity of the second body	speed / second_final	`m/s`	
`m1`	mass of the first body	mass / first	`kg`	
`m2`	mass of the second body	mass / second	`kg`	
`v1`	initial velocity of the first body	speed / first_initial	`m/s`	
`v2`	initial velocity of the second body	speed / second_initial	`m/s`	

- **Conditions:** elastic_collision
- **Applies when:** two bodies colliding elastically along a line
- **Fails when:** inelastic collisions; glancing collisions
- **Assumes:** a head-on (one-dimensional) collision; kinetic energy conserved; no external forces
- **Evidence:** theorem

```
mechanics_ai999x.solve('elastic_collision_second_final', m1='2 kg', m2='1 kg', v1='3 m/s', v2='0
m/s', target='v2f')
# -> 4 m/s   (SOLVED)
```

$$v_{2f} = \frac{(m_2 - m_1)v_2 + 2m_1v_1}{m_1 + m_2}$$
$$v_{2f} = \frac{2m_1v_1 - m_1v_2 + m_2v_2}{m_1 + m_2} = 4\ \mathrm{m/s}$$

invalid call	outcome	message
without `m1`	NEEDS_INPUT	elastic_collision_second_final needs m1 (mass of the first body, kg). Example: solve('elastic_collision_second_final', m1='... kg', m2='...
`m1` in a wrong unit	INVALID_INPUT	m1 is a mass: its unit must be like kg, not 's'.

Questions in words that work:

- "In an elastic collision the first mass is 2 kg with a velocity of the first of 3 m/s, and the second mass is 1 kg with a velocity of the second of 0 m/s. What is the final velocity of the second?" → 4 m/s
- "Two carts collide elastically: the first mass is 1 kg and the velocity of the first is 4 m/s; the second mass is 3 kg and the velocity of the second is -2 m/s. Find the final velocity of the second." → 1 m/s

Velocity change of a rocket (Tsiolkovsky) — `rocket_delta_v`

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$

input	meaning	kind / role	unit	default
`dv`	change in velocity	speed / delta_v	`m/s`	
`ve`	exhaust velocity	speed / exhaust	`m/s`	
`m0`	initial mass	mass / initial	`kg`	
`mf`	final mass	mass / final	`kg`	

- **Conditions:** rocket
- **Applies when:** a rocket burning fuel in free space
- **Fails when:** gravity losses; atmospheric drag
- **Assumes:** no gravity or drag during the burn; a constant exhaust velocity
- **Evidence:** theorem

```
mechanics_ai999x.solve('rocket_delta_v', ve='3000 m/s', m0='1000 kg', mf='400 kg', target='dv')
# -> 2748.87 m/s   (SOLVED)
```

$$\Delta v = v_e \ln\left(\frac{m_0}{m_f}\right)$$
$$\Delta v = \ln\left(\left(\frac{m_0}{m_f}\right)^{v_e}\right) = 2748.87\ \mathrm{m/s}$$

invalid call	outcome	message
without `ve`	NEEDS_INPUT	rocket_delta_v needs ve (exhaust velocity, m/s). Example: solve('rocket_delta_v', ve='... m/s', m0='... kg', mf='... kg').
`ve` in a wrong unit	INVALID_INPUT	ve is a speed: its unit must be like m/s, not 'kg'.

Questions in words that work:

- "A rocket with an exhaust velocity of 3000 m/s has an initial mass of 1000 kg and a final mass of 400 kg after the burn. What is its change in velocity?" → 2748.87 m/s
- "Find the delta-v of a rocket whose exhaust speed is 4400 m/s, with an initial mass of 50000 kg and a final mass of 10000 kg." → 7081.53 m/s